

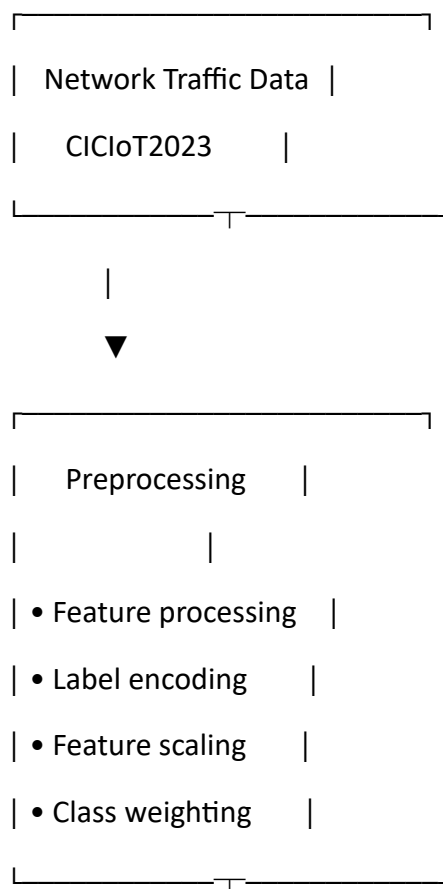
Absolutely. Since your implementation and deployment are now finished, we can build the **complete final-year-project package** around the actual system and results rather than inventing anything.

I'd suggest we prepare these in this order:

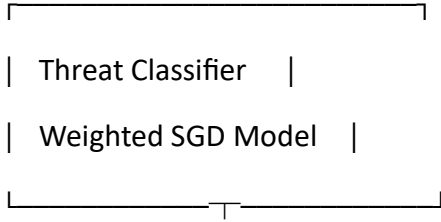
1. **System Architecture Diagram**
2. **Final Report content**
3. **Methodology**
4. **Results & Discussion**
5. **Limitations & Future Work**
6. **Final Presentation/PPT structure**
7. **Viva questions and answers**
8. **Live demo script**

1. System Architecture

Your final architecture should communicate this flow:



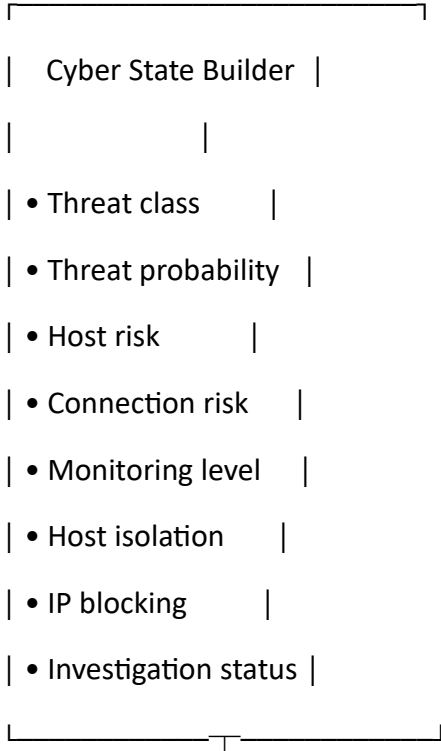
|



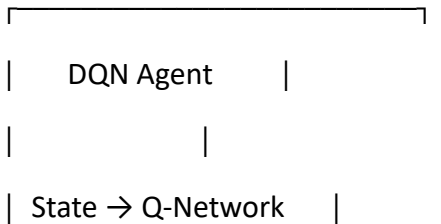
|

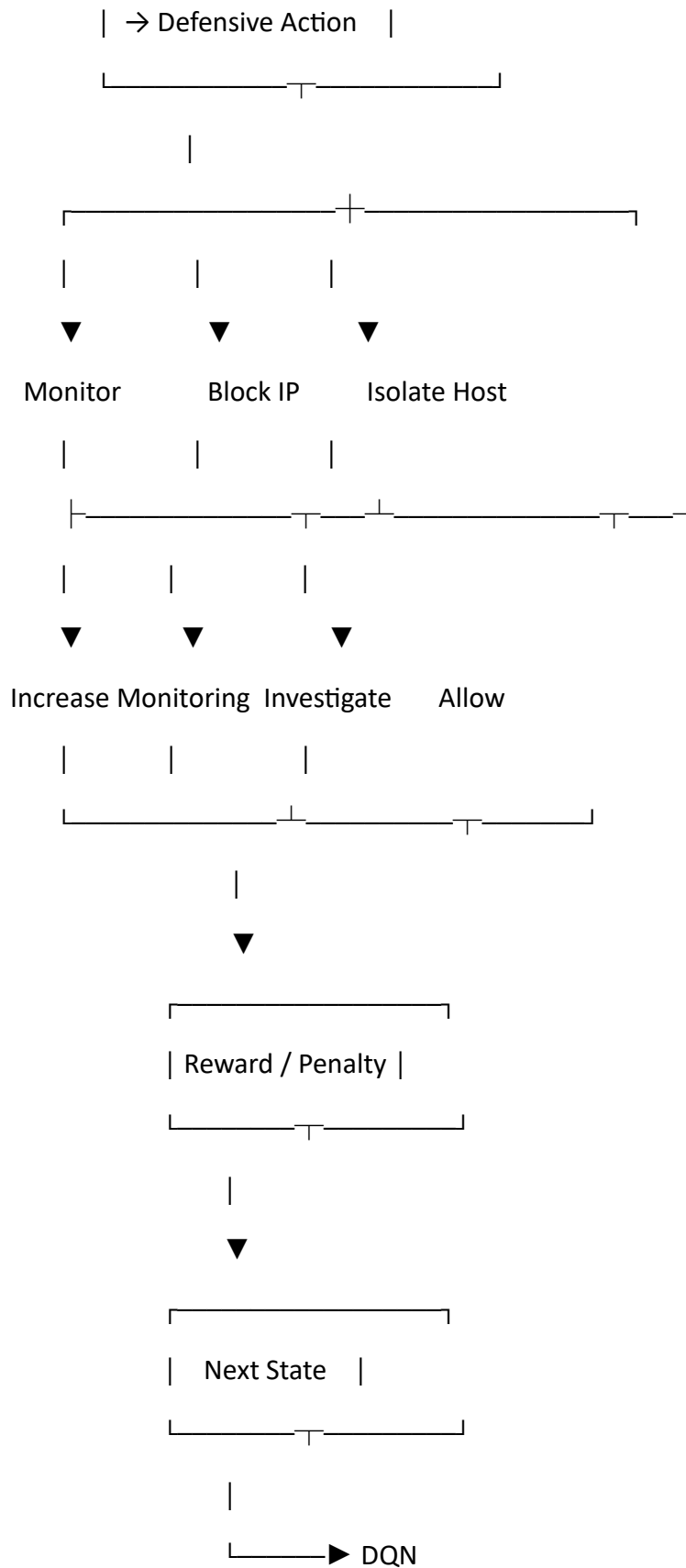
Threat class + confidence

|



|





2. Final Report Structure

Use this structure for your final report.

Chapter 1 — Introduction

Title: Autonomous Cyber Threat Hunting Using Machine Learning and Deep Reinforcement Learning

1.1 Background

Modern networks generate large volumes of traffic, making manual monitoring and incident response difficult. Cybersecurity systems therefore require automated mechanisms capable of identifying malicious activity and selecting appropriate responses.

Traditional intrusion detection systems generally focus on detecting or classifying attacks. However, detection alone does not determine what defensive action should be taken. This project combines supervised machine learning with deep reinforcement learning to address both stages.

1.2 Problem Statement

The project addresses the problem of developing an autonomous cyber-threat-hunting system that can:

- identify network threats,
- estimate threat confidence,
- construct a representation of the current cyber state,
- select an appropriate defensive response,
- learn response policies through reinforcement learning, and
- provide an interactive monitoring interface.

1.3 Objectives

1. Preprocess network traffic data for machine-learning analysis.
2. Develop a multiclass threat-classification model.
3. Handle class imbalance using class weighting.
4. Design a cyber-security environment for reinforcement learning.
5. Develop a Deep Q-Network agent for response selection.
6. Evaluate the agent using predefined response criteria.
7. Develop an interactive dashboard for system visualization.

8. Deploy the completed application.
-

3. Methodology

3.1 Dataset

The project uses the **CICIoT2023** network-traffic dataset.

The processed dataset contains:

- **46 input features**
- **34 threat classes**
- a target label column.

The classes include categories such as:

- BenignTraffic
 - DDoS attacks
 - DoS attacks
 - Mirai attacks
 - Reconnaissance attacks
 - DNS spoofing
 - ARP spoofing
 - SQL injection
 - XSS
 - command injection
 - vulnerability scanning
 - uploading attacks
 - browser hijacking
 - backdoor malware
-

3.2 Data Preprocessing

The preprocessing pipeline performs:

Raw network traffic



Feature inspection



Data cleaning



Label encoding



Feature scaling



Class-weight calculation



Train / validation / test datasets

The original large datasets are used for training and evaluation, while a separate **5,000-row deployment dataset** is used by the deployed dashboard.

4. Threat Classification

The classifier uses an SGDClassifier with logistic loss.

Conceptually:

Network features



Scaled feature vector



SGD classifier



Predicted threat class



Class probability/confidence

The weighted model was used because the dataset contains significant class imbalance.

Test performance

Metric	Result
Accuracy	78.75%
Macro Precision	50.78%
Macro Recall	42.66%
Macro F1	42.92%
Weighted Precision	78.45%
Weighted Recall	78.75%
Weighted F1	74.91%

How to explain this in the viva

Say:

"The model achieved 78.75% overall test accuracy. However, the macro F1 score was substantially lower because the dataset is highly imbalanced and several rare attack classes have limited representation. Therefore, accuracy alone is not sufficient to characterize the classifier's performance."

That's a much stronger answer than simply claiming the classifier is highly accurate.

5. Reinforcement Learning Environment

The classifier's output is converted into an eight-dimensional cyber state.

State =

[
threat_class,
threat_probability,
host_risk,
connection_risk,
monitoring_level,
host_isolated,
ip_blocked,

investigation_active

]

Therefore:

State size = 8

The DQN has **6 possible actions**:

ID Action

0 Monitor

1 Block IP

2 Isolate Host

3 Increase Monitoring

4 Investigate

5 Allow

6. DQN Architecture

Your network is:

Input

8

|



Linear Layer

128 neurons

|

ReLU

|



Linear Layer

128 neurons



The network contains:

18,438 trainable parameters.

The DQN learns:

$$Q(s, a)$$

where:

- s = current cyber state
- a = defensive action
- $Q(s, a)$ = estimated future value of taking action a

The agent selects the action with the highest Q-value during deterministic evaluation.

7. Reward Design

The reward function is threat-dependent.

For example, for high-severity attacks:

Isolate Host	+10
Block IP	+8
Investigate	+5
Increase Monitoring	+3
Monitor	-5
Allow	-10

For benign traffic:

Allow	+5
Monitor	+4

Investigate -2

Increase Monitoring -2

Block IP -6

Isolate Host -8

This encourages the agent to respond strongly to dangerous traffic while avoiding unnecessary defensive actions against benign traffic.

8. DQN Training Results

Training configuration:

Parameter	Value
Episodes	20
Steps/episode	500
Total environment steps	10,000
Batch size	64
Replay buffer	50,000
Discount factor γ	0.99
Initial ϵ	1.0
Minimum ϵ	0.05
Learning rate	0.001
Target update frequency	5 episodes

Training completed successfully.

Best episode reward:

1413

Final episode reward:

1335

Final epsilon:

0.9046

Important interpretation

Do **not** say:

"The DQN fully converged."

Instead say:

"The DQN training completed successfully and demonstrated effective defensive-action selection on the evaluation subset. However, the relatively high final epsilon indicates that the training configuration remained exploratory, so the results should not be interpreted as proof of globally optimal convergence."

That is technically defensible.

9. Final RL Evaluation

Your independent 1,000-sample evaluation produced:

Metric	Result
Samples	1,000
Classification accuracy	78.80%
Action appropriateness	98.60%
Average reward	8.549
Missed threats	0
Missed-threat rate	0%
Unnecessary defensive actions	14
Unnecessary-action rate	1.40%

The evaluation subset was attack-heavy:

- 715 HIGH
- 255 MEDIUM
- 10 INVESTIGATION
- 20 LOW

Therefore, the 98.6% action-appropriateness figure should specifically be described as:

Action appropriateness under the project's predefined criteria on the evaluated 1,000-sample subset.

Don't present it as general real-world response accuracy.

10. Limitations

This section is important.

10.1 Class imbalance

Several rare classes have weak recall.

Examples include:

- Backdoor_Malware
- BrowserHijacking
- CommandInjection
- DDoS-HTTP_Flood
- DDoS-SlowLoris
- Recon-PingSweep
- SqlInjection
- Uploading_Attack
- XSS

10.2 Offline evaluation

The current system evaluates recorded network observations rather than operating against a live production network.

10.3 Simulated defensive actions

Actions such as:

- Block IP
- Isolate Host
- Investigate

are represented within the simulation environment rather than actually modifying firewall or endpoint configurations.

10.4 Reward dependency

The RL agent's behavior depends on the manually designed reward function.

Changing the reward structure can change the learned policy.

10.5 Limited training duration

Only 10,000 environment steps were used for the current DQN training run, and the final epsilon remained high.

11. Future Work

Your future-work section can include:

Real-time traffic integration

Integrate the system with live packet/network telemetry.

Automated response integration

Connect actions to:

- firewalls
- endpoint isolation systems
- SIEM platforms
- SOAR platforms

Improved class imbalance handling

Investigate:

- focal loss
- resampling
- synthetic data generation
- specialized rare-class models

Advanced RL algorithms

Future versions could investigate:

- Double DQN
- Dueling DQN
- Prioritized Experience Replay
- PPO

- multi-agent reinforcement learning

Continuous learning

Allow the system to incorporate newly observed attack patterns and adapt over time.

Explainable AI

Provide explanations for:

Why was this traffic classified as malicious?

Why did the agent choose Block IP?

Why was the threat considered high risk?

12. Final PPT Structure

I recommend around **12–14 slides**.

Slide 1 — Title

Autonomous Cyber Threat Hunting Using Machine Learning and Deep Reinforcement Learning

Name, roll number, department, guide, institution.

Slide 2 — Introduction

- Growth of network threats
- Limitations of manual monitoring
- Need for automated detection and response

Slide 3 — Problem Statement

Show:

Network Traffic



Threat Detection



Threat Classification



Response Selection

Explain that conventional IDS focuses primarily on detection.

Slide 4 — Objectives

Use the 6–8 objectives listed earlier.

Slide 5 — Dataset

Show:

CICIoT2023

↓

46 features

↓

34 classes

↓

Preprocessing

↓

Train / Validation / Test

Slide 6 — System Architecture

Use the architecture diagram above.

Slide 7 — Threat Classification

Show:

- Weighted SGD classifier
- 46 features
- 34 classes
- 78.75% test accuracy
- Macro F1 = 42.92%

Slide 8 — Reinforcement Learning

Show:

Cyber State

↓

DQN



Q-values



Defensive Action



Reward



Next State

Slide 9 — DQN Architecture

Show:

8 → 128 → 128 → 6

and list the six actions.

Slide 10 — Results

Use the 1,000-sample evaluation results.

Highlight:

98.60% action appropriateness

8.549 average reward

0 missed threats

Slide 11 — Dashboard

Use screenshots from your deployed application.

Show:

- threat classification
- confidence
- risk
- selected response
- Q-values
- action history
- statistics

Slide 12 — Limitations

Keep it honest:

- class imbalance
- offline dataset
- simulated responses
- limited training
- reward dependency

Slide 13 — Future Scope

Use the future-work items.

Slide 14 — Conclusion

A concise conclusion:

The project demonstrates an integrated framework combining supervised machine learning for cyber-threat classification with deep reinforcement learning for automated response selection. The system successfully classifies network observations, represents the resulting cyber state, selects defensive actions through a DQN agent, and provides an interactive dashboard for monitoring and analysis.

13. Viva — Questions You Should Prepare

Q1. Why did you use both machine learning and reinforcement learning?

Answer:

"The supervised classifier answers the detection problem—what type of threat is present. The reinforcement-learning agent addresses the decision problem—what defensive action should be taken given the current cyber state."

Q2. Why DQN?

"The response space consists of a finite set of discrete defensive actions, which makes Deep Q-Learning suitable for estimating the value of each action for a given cyber state."

Q3. What is the state?

"The state is an eight-dimensional representation containing threat class, threat probability, host risk, connection risk, monitoring level, host isolation status, IP blocking status, and investigation status."

Q4. What are the actions?

"Monitor, Block IP, Isolate Host, Increase Monitoring, Investigate, and Allow."

Q5. What is the reward?

"The reward represents the desirability of an action for the current threat category. Appropriate defensive actions receive positive rewards, while inappropriate or unnecessarily aggressive actions receive penalties."

Q6. Why is the classifier accuracy higher than macro F1?

"Because the dataset is imbalanced. High-frequency classes contribute heavily to overall accuracy, while macro F1 gives every class equal importance and therefore exposes the weaker performance on rare classes."

Q7. Is your system really autonomous?

A careful answer:

"Within the project environment, the response-selection process is autonomous because the DQN selects the defensive action without manual selection. However, the current implementation simulates the defensive actions rather than executing real firewall or endpoint changes."

Q8. What is the main limitation?

"The major limitations are class imbalance, offline dataset evaluation, simulated defensive actions, and limited DQN training duration."

Q9. What happens if the classifier makes a wrong prediction?

"The DQN receives the classifier-derived state, so classification errors can propagate into the response-selection stage. This is an important limitation and motivates future work on uncertainty-aware and jointly optimized detection-response systems."

Q10. What is your project's main contribution?

"The project integrates threat classification with reinforcement-learning-based response selection in a single cyber-threat-hunting workflow, rather than treating detection and response as completely separate processes."

14. Live Demo Script

For your final presentation, keep the demo short.

Step 1

Open the deployed dashboard.

Say:

"This is the deployed interface of the autonomous cyber-threat-hunting system."

Step 2

Process a sample.

Point out:

- threat class
- confidence
- risk level
- selected RL response

Step 3

Open/show Q-values.

Say:

"The DQN estimates a Q-value for each available defensive action. The action with the highest estimated value is selected."

Step 4

Process several samples.

Show the history changing.

Step 5

Show the statistics.

Point out:

- samples processed
- threats detected
- average confidence
- average reward
- action distribution

Step 6

Finish with:

"The important distinction is that the classifier identifies the threat, while the reinforcement-learning agent determines the response."

That's the key sentence I'd make sure you remember for the viva.

Yes — **5,000 samples is the deployed dashboard dataset**, but the **1,000 samples refers to your independent DQN evaluation**. They are two different things.

Keep these two numbers separate

Purpose	Samples	What it means
DQN evaluation	1,000	Formal evaluation used to calculate the RL results
Deployed dashboard	5,000	Demo/deployment dataset available for users to process

So the results I gave you earlier remain:

Independent DQN evaluation: 1,000 samples

while your deployed application now uses:

Deployment dataset: 5,000 samples

For your final PPT/report

Don't write:

"The system was evaluated on 5,000 samples and achieved 98.6%..."

because the 98.6% result came from the **1,000-sample evaluation**, not the 5,000-row deployment dataset.

Instead write:

Deployment: A 5,000-sample processed dataset was prepared for the deployed Streamlit application to provide a larger demonstration dataset.

And:

RL Evaluation: The trained DQN agent was independently evaluated on 1,000 samples, achieving 98.60% action appropriateness under the project's predefined evaluation criteria.

One more important distinction

Your deployed dashboard can process **up to 5,000 available samples**, but that doesn't automatically mean we've generated new performance metrics for all 5,000.

If you want your **final report to say "evaluated on 5,000 samples"**, we would need to actually run the evaluation over the 5,000-row deployment dataset and generate the corresponding metrics.

I would recommend doing that before finalizing the report, because it would give you a clean **5,000-sample final evaluation** matching the deployed version.